

APIs, middleware, and cloud). These changes may then require additional resources and personnel to perform. For more information, refer to the *FFIEC IT Handbook*'s "[Information Security](#)" and "[Outsourcing Technology Services](#)" booklets.

IV.C Licenses, Agreements, and Copyright Protection

Systems and components are often licensed, not purchased. Typically, licenses do not give a client (sometimes referred to as a licensee) ownership rights over the licensed product. The licensor³⁵ provides licenses to clients that grant certain rights to use the system or components. The licensor typically grants rights to a client for a specific duration and may charge annual and other fees for system or component use. An effective license and agreement review process at the entity is important to ensure that terms are clearly defined and understood prior to engagement. Before negotiating licenses, management should accurately assess current and future needs and ensure that licenses will continue to meet the entity's needs.

The defined scope of a license is a key issue in evaluating and, when feasible, negotiating the terms of a licensing agreement. When reviewing a license, management should confirm that it clearly states whether system or component usage is exclusive, the number of user licenses, and whether there are any time, place, manner, or other types of limitations with the system or component's use. Effective management reviews licenses to determine whether they allow backup copies related to mission-critical systems or components on which the entity may need to rely for disaster recovery or business continuity purposes. The entity may need to negotiate with licensors to ensure the ability to backup hardware and software if backups are not provided for in the license. In difficult license agreement negotiations, including when an entity has limited negotiating power, management should understand any resulting limitations and consequent risks. Possible actions that an entity might take in such circumstances include determining whether the product or service can still meet the entity's needs, whether the product or service would result in increased risk to the entity, and whether residual risks are acceptable. If the license agreement is unacceptable for the entity, management may consider other approaches, such as employing other third parties or conducting the activity in-house. In certain circumstances, entities may gain an advantage by negotiating license agreements as a group with other entities (e.g., user groups and banking associations).³⁶

If the entity plans to provide the systems or components to other related entities (e.g., subsidiaries or contractors), management should include those entities as users in the licenses. If management is responsible for developing systems and components and providing them to other entities or its subsidiaries, management should implement appropriate licenses for the systems or components developed.

Knowledge of license expiration dates is crucial for the planning and management of license renewal. If there is no license expiration or the license renews automatically (i.e., a perpetual

³⁵ A "[licensor](#)" is defined as "the person who gives or grants a license."

³⁶ For more information, refer to [Interagency Guidance on Third-Party Relationships: Risk Management](#). Specific agency references to this document include ([FDIC](#)) [FIL 29-2023](#), ([FRB](#)) [SR Letter 23-4](#), and ([OCC](#)) [Bulletin 2023-17](#).

license), the license agreement should explicitly outline those terms. Failure to specify a fixed term or termination date does not automatically provide the entity with a perpetual license. Management should specify in its agreement the appropriate time periods for all licenses and the minimum amount of notice required for license termination. Effective licensing and ITAM processes include tracking license time frames.

Effective management periodically reviews system and component licenses to compare all installed licensed systems and components with the respective license terms. This allows management to detect unauthorized installations and validate appropriate license use. If there is a discrepancy between the total number of licenses allowed by the agreement and the installed software, management should address the inappropriate usage. Audit personnel should consider this when developing the scope of their reviews.

There are several other types of agreements management should consider and review in addition to licenses. For example, a third party may offer maintenance agreements, which outline available maintenance services (e.g., provision of new versions, releases, or updates). Another type of agreement is a development agreement, which outlines terms for the development of systems or components. This agreement may be internal or with a third party. If the agreement involves a third party, the entity may not retain ownership rights even if the entity paid to have the system or component developed.

A license or agreement should address the availability and cost of system and component updates and modifications (i.e., maintenance). When drafting agreements, effective management determines whether a third-party vendor provides access to source or object code (“code”). Management should have the vendor’s permission and participation for modifications to the code for systems and components. Unauthorized modifications to the code may void maintenance agreements.

A license or agreement should direct third-party vendors to deliver appropriate documentation. This should include application and user documentation. Management should ensure that the agreement specifies that updated application and user documentation will be provided when any changes are made to procured systems or components.

IV.C.1 Software Licenses

Effective ITAM includes software license oversight. Software licenses have varying provisions and requirements for ownership and usage. It is important to consider license provisions and all consequences for violating provisions. In more critical instances (i.e., critical to providing business services), entities may engage specialized legal expertise to understand and mitigate software license risks. If an entity develops software and licenses that software to others, management should be aware of the liabilities that come with licensing activities, including considerations related to software maintenance, integration, compatibility, and fraud. Tracking and managing the usage and implementation of different types of licenses for which an entity paid helps ensure that the entity is operating within the licenses’ specific requirements. There are several types of software licenses, including free and open-source software licenses and proprietary licenses.

IV.C.1(a) *Free and Open-Source Software Licenses*

There are two types of free and open-source software (FOSS) licenses:

- **Public domain (and equivalent) license:** Public domain software is software not protected by copyright laws of any nation that may be freely used without permission of or payment to the creator and that carries no creator warranties.³⁷ This reduces creator liability relative to the software's use. Software creators may renounce software ownership and any usage limitations to ensure that the software is able to be used by anyone regardless of jurisdiction license requirements and to limit their liability.³⁸
- **Open-source license:** Typically, open-source software used by entities has some type of attached license. Companies often license open-source software through a third party that makes software use and maintenance easier. Just as with proprietary software licenses, it is important to understand the license provisions such as who owns the software source code, and any restrictions on what can be done with the software. Examples of open-source software license subtypes are lesser general public licenses, permissive licenses, general public licenses, and copyleft licenses.³⁹

FOSS is typically acquired and used in an as-is manner. An entity may not be aware of the developer(s) in public domain FOSS. There may or may not be updates made available for the software, and changes made to the software may be made by other individuals besides the developer. Risks associated with FOSS include the potential for unidentified or unpatched vulnerabilities and development changes by unknown parties. An accurate inventory of FOSS should be maintained as part of an effective ITAM process, and management should understand and abide by the requirements of FOSS licenses. Additionally, some licenses require users to contribute software additions or enhancements, or they may prohibit the use for commercial purpose or profit. Management should be aware of what information is shared when contributing an enhancement to the FOSS development community.

IV.C.1(b) *Proprietary Software Licenses*

In proprietary software licenses, the software publisher retains software ownership, and the entity and end users agree to accept and abide by the terms of the software licenses. A licensee and its employees may only use the software as allowed by the license and typically cannot copy, modify, or distribute the software and its underlying code. An end user license agreement (EULA) provides specifics regarding provisions or restrictions. EULAs typically apply to for-profit open-source software and non-open-source software, and they are often used for commercial purposes. Some examples of proprietary licenses include noncommercial use-only

³⁷ Refer to [NIST Glossary](#).

³⁸ For more information, refer to public-domain equivalent licenses, such as Creative Commons' [CC0](#).

³⁹ Copyleft is a general licensing method for making a program free while requiring all modified and extended versions of the program to be free as well. There are strong and weak copyleft licenses. Refer to Department of Defense (DOD), [DoD Open Source Software FAQ](#).

licenses, proprietary licenses, and trade secret licenses. Regardless of the types of licenses encountered, they often cover the following:

- Ownership rights of the software.
- Permission to use the software and usage rights.
- Number of users allowed to use the software.
- Inclusions or add-ons (e.g., support, maintenance, and software upgrades).
- Warranty terms.
- Installation location and frequency.
- Permissions and limitations regarding copying, modifying, and distributing software and the underlying code.
- Copyrights.
- Terms for license termination.
- Software performance guarantees.
- Penalties and fees for noncompliance.
- Length of the agreement and its terms.

For more information on ITAM, which includes management of software licenses, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" booklet.

IV.C.2 Hardware Licenses

Entities identify and manage hardware licenses in ITAM programs to ensure that their hardware use is consistent with agreements between the manufacturer, any third party, and the entity. Entities may access components (e.g., circuit boards, servers, switches, tokens, universal serial buses, mobile devices, service ports, and cameras) in solution development. Hardware licenses, like software licenses, should identify any limitations on hardware use.

If an entity licenses COTS hardware, the hardware may come as is and any alterations or modifications may void the warranty in the license. If the hardware is proprietary, items such as warranties, modifications, and maintenance should be included in the license agreement. If an entity builds hardware and then licenses that hardware to other entities, effective management maintains awareness of the liabilities that come with licensing activities, including considerations related to hardware maintenance, integration, compatibility, and fraud.

For more information on ITAM, which includes management of hardware and software licenses, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" booklet.

IV.C.3 Copyright Protection

Copyright laws⁴⁰ protect proprietary as well as open-source software. Examples of elements covered by copyright protections generally include the following:

⁴⁰ Refer to U.S. Copyright Office, [Copyright Law of the United States \(Title 17\)](#).

- **Reproduction:** The copyright owner maintains rights in any reproduction of the original work.
- **Adaptation:** The copyright owner has rights to adaptations based on the original work.
- **Publication:** The copyright owner has the right to distribute the work to the public.
- **Performance:** The copyright owner has the right to perform the work publicly.
- **Display:** The copyright owner has the right to display the work publicly.

Unlicensed system or component use or licensing agreement violations expose entities to litigation. Common challenges in deploying software, consistent with licenses and copyrights, include keeping track of deployed instances when the entity is large and decentralized, keeping track of deployed instances when deployments are made to virtual environments and are not automatically tracked, instances being shared when the license does not provide for sharing, and license limits on concurrent system or component use.

Measures that management may employ to protect against copyright violations include obtaining a site license that authorizes system or component use at all of the entity's locations, informing users of the rules governing site licenses, and acquiring an automated program that scans for unauthorized system or component use or copyright violations. While these measures may help identify copyright violations, the best control mechanism is a well-communicated corporate policy that management enforces and auditors validate for compliance.

IV.D Secure Development

Secure development is an approach to creating products and services that incorporates security into every phase of the product's or service's development and use. Secure software development includes practices "to reduce the number of vulnerabilities in released software, to reduce the potential impact of the exploitation of undetected or unaddressed vulnerabilities, and to address the root causes of vulnerabilities to prevent recurrences."⁴¹ While this booklet primarily focuses on the secure development of software, there are also techniques for securely developing hardware.⁴² System, component, and service development, acquisition, and maintenance inherently presents security risks. This is true whether the system, component, or service is developed internally or acquired.

When management outsources development to third parties, it should monitor the third parties' conformance to contract requirements, Information Security Standards,⁴³ and other legal and

⁴¹ Refer to NIST SP 800-218, [Secure Software Development Framework \(SSDF\) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities](#).

⁴² For more information on mitigation of hardware development risks, refer to DOD's [Securing Defense-Critical Supply Chains](#).

⁴³ Refer to 15 USC 6801 and 6805(b), [GLBA](#), further implemented by FFIEC members as follows: FDIC: 12 CFR 364, appendix B, "[Interagency Guidelines Establishing Information Security Standards](#)," and 12 CFR 364, supplement A to appendix B, "[Interagency Guidance on Response Programs for Unauthorized Access to Customer Information and Customer Notice](#)"; FRB: Regulation H, 12 CFR 208, appendix D-2, "[Interagency Guidelines Establishing Information Security Standards](#)," and Regulation Y, 12 CFR 225, appendix F, "[Interagency Guidelines Establishing Information Security Standards](#)"; NCUA: 12 CFR 748, appendix A, "[Guidelines for Safeguarding](#)

regulatory requirements. A third party providing development services should maintain policies, standards, and procedures that promote secure development consistent with the entity's requirements and the board's risk appetite. When acquiring systems and components, management should evaluate the third party's secure coding standards (e.g., through independent certification or audit) and consider related supply chain risks.⁴⁴ It is also important that third-party system or component developers use secure coding standards, that its use is independently verified, and that management reviews the verification. This is important to verify when products are initially purchased and to review periodically as products are updated and patched. Furthermore, system maintenance should consider security impacts of integration with legacy systems and components. For more information, refer to the *FFIEC IT Handbook's* "[Outsourcing Technology Services](#)" and "[Information Security](#)" booklets.

General software quality management practices contribute to secure systems and components. For example, when thorough testing results in functional flaws being corrected, security vulnerabilities can also be identified and corrected. There are quality assurance tests that can specifically identify security flaws such as fuzz testing⁴⁵ and penetration testing. There are also quality management tools such as secure code reviews and source code security analyzers⁴⁶ that can systematically and efficiently identify security vulnerabilities and other flaws for remediation. Entities that use these practices and tools are more likely to produce secure systems and components than those that do not.

Manual or automated code reviews help identify vulnerabilities. Manual code reviews⁴⁷ are performed by entity personnel or a third party and may be less costly than an automated code review.⁴⁸ However, limitations on reviewers' knowledge or experience may result in unidentified issues, such as syntax errors and code vulnerabilities due to human error. Additionally, manual reviews may not be effective for extensive volumes of testing. Automated code reviews, on the other hand, may be more sophisticated, can reduce human error, and generally provide more timely identification of weaknesses. These tools are scalable and can be continuously updated as technology and vulnerabilities evolve. Entities review software code that has been implemented to mitigate the risk that vulnerabilities may be introduced through routine updates and patches. If

[Member Information](#)"; and OCC: 12 CFR 30, appendix B, "[Interagency Guidelines Establishing Information Security Standards](#)."

⁴⁴ For more information on supply chain risks affecting development, refer to the "[Development](#)" and "[Supply Chain Considerations](#)" sections of this booklet.

⁴⁵ Fuzz testing refers to a black box software testing technique, which basically consists of finding implementation bugs using malformed or semi-malformed data injection in an automated fashion by tools referred to as "fuzzers," which are programs or scripts that submit some combination of inputs to the test target to reveal how it responds. For more information, refer to OWASP's [Fuzzing](#).

⁴⁶ According to NIST, "Source code security analyzers examine source code to detect and report weaknesses that can lead to security vulnerabilities."

⁴⁷ Manual code review is the process of personnel reviewing the source code line by line to identify possible vulnerabilities.

⁴⁸ Refer to NIST Internal Report (IR) 8397, [Guidelines on Minimum Standards for Developer Verification of Software](#).

an automated code review process is implemented, management should ensure that the embedded rules of the code review tools are appropriately configured and used. Management selects code review types that are effective for the entity's project and available resources, and management validates that the code review process meets its needs.

System vulnerability scanning is another tool for mitigating security risks during development.⁴⁹ Scanning software is implemented in development environments so that vulnerabilities are identified and remediated before they are exposed to users and exploited by malicious actors. Requirements for system vulnerability scanning can be included in contracts and ancillary agreements if the underlying system is critical enough. Entities also continue to scan systems for vulnerabilities once systems are deployed and being maintained to mitigate the risk that vulnerabilities are introduced as the system is upgraded. Refer to *FFIEC IT Handbook's* "[Information Security](#)" and "[Architecture, Infrastructure, and Operations](#)" booklets.

IV.E Data

Mitigating the risk of data compromise is important in addition to controlling software and system development. To mitigate the risk of data compromise, entity personnel establish and maintain data inventories that include data characteristics such as storage location, criticality, and sensitivity. They also implement and maintain controls that protect the data consistent with their characteristics. For example, an entity may implement a policy that highly sensitive data will not be used in development and test environments. Instead, an entity may create synthetic data to use in development and testing that is a proxy for the actual, highly sensitive data. Another control example is documenting system data use so that security engineers can implement controls that protect data throughout process execution.⁵⁰ For more information, refer to the *FFIEC IT Handbook's* "[Information Security](#)" and "[Architecture, Infrastructure, and Operations](#)" booklets.

IV.F Secure Operating Environments

Management should maintain a secure operating environment throughout development, acquisition, and maintenance activities whether entity employees are operating the system or a third party's employees are operating the system. Entities use various controls to secure operating environments, such as configuration management, access control, and business continuity planning and testing. Effective application of formal policies addressing the least privilege principle help control access to systems, components, and data. An effective entity establishes documented policies to communicate and implement operating environment controls and provides for periodic audits for validation of the controls to determine whether systems and

⁴⁹ Like network port and service identification, vulnerability scanning identifies hosts and host attributes (e.g., operating systems, applications, or open ports), but it also attempts to identify vulnerabilities rather than relying on human interpretation of the scanning results. Refer to NIST SP 800-115, [Technical Guide to Information Security Testing and Assessment](#).

⁵⁰ NIST states in SP 800-37, rev. 2, [Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy](#), that "such documentation includes, for example, data flow diagrams, entity relationship diagrams, database schemas, and data dictionaries."

components are functioning as intended. Additional operating environment control examples are listed below:

- Identifying personnel allowed to access development, test, and production environments.
- Determining under which conditions they may access systems, components, and data within production environments.
- Maintaining authorization procedures for granting and verifying access.
- Determining how long users require access.
- Segregating operating environments (e.g., development, test, QA, and production) and specifying access controls for each operating environment, while accounting for new business functions.
- Ensuring resilience when developing, acquiring, or maintaining systems, components, services, and data.
- Maintaining control over remote access (e.g., timing, location, personnel, and multifactor authentication [MFA]).
- Providing appropriate controls over access to and by specific devices.
- Maintaining business line and IT representation when determining security and resilience measures.
- Maintaining ITAM processes for current and planned assets.

For more information, refer to the *FFIEC IT Handbook's* “[Architecture, Infrastructure, and Operations](#),” “[Outsourcing Technology Services](#),” and “[Information Security](#)” booklets.

IV.G Microservices

Microservices refers to an architectural and organizational approach to building systems that involves much smaller components than historical approaches and requires attention to mitigate risks that could be material. NIST defines “microservices” as “a set of containers that work together to compose an application.” This includes a series of interconnected containers or applications working in coordination (referred to as “orchestration”). Microservices-based architectures and applications are prevalent in cloud-based systems but can also be run in non-cloud-based virtualized environments.

Traditionally developed, or monolithic,⁵¹ applications comprise multiple lines of code developed as a single application. To work properly, these applications reside on a single server or virtual machine. Any updates rely on an update to the entire application. By contrast, microservices-based applications consist of small, self-contained (i.e., containers), independent services that use APIs to communicate and work together. Each microservice has a singular purpose (e.g., messaging, calculations, or database access). Microservices are generally technologically heterogeneous and can be written in different languages, use different development platforms, and be spread across multiple servers or virtual machines. Because microservices architecture has independent containers working together to create an application, updates can be applied to any one independent container without having to update the entire application. Additionally,

⁵¹ A monolithic application (or [monolith](#)) refers to a single unit, as opposed to a microservice architecture, which comprises smaller, independently deployable services.

each microservice can be reused to create new applications. These properties of microservices facilitate scalability and a faster, more agile application development process.

During acquisition or development of systems using microservices, effective control practices are used to mitigate the risk of systems malfunctioning and risk of security incidents. An example of an effective control is deploying and using highly secure and stable service registries for tracking the services included in a system. A microservice has two broad functions:⁵²

- Business logic, which implements the business functionalities, computations, and service composition or integration logic.
- Network processes, which manage the interservice communication mechanisms and are built on top of the underlying OS-level network stack.

NIST provides four drivers underlying the design principles of microservices:⁵³

- Each microservice must be managed, replicated, scaled, upgraded, and deployed independently of other microservices.
- Each microservice must have a single function and operate in a bounded context (i.e., have limited responsibility and dependence on other services).
- All microservices should be designed for constant failure and recovery, and therefore must be as stateless⁵⁴ as possible.
- One should reuse existing trusted services (e.g., databases, caches, and directories) for state management.

Additionally, NIST provides a list of the following design principles for microservices:

- **Autonomy:** Ability to operate independently as a self-contained entity that delivers all of the functions of an operations stack.
- **Loose coupling:** Minimal dependency between services so the change in one service does not require a change in another service.
- **Reusability:** Use of a service in different ways for different purposes.
- **Composability:** Ability to be assembled easily and configured to meet specific needs.
- **Fault tolerance:** Property of a system that allows proper operation even if components fail.
- **Discoverability:** Ability to be located and identified using discovery services.
- **Alignment of APIs with business processes:** Aligning microservices, APIs, and business processes during the design phase allows for easier changes to the services when business processes change.

⁵² Refer to NIST SP 800-204A, [Building Secure Microservices-Based Applications Using Service-Mesh Architecture](#).

⁵³ Refer to NIST SP 800-204, [Security Strategies for Microservices-Based Application Systems](#).

⁵⁴ Refer to [NIST Glossary](#).

Because microservices can be replicated or deployed across servers, a microservice architecture generally has a directory for microservices to publish their locations, also referred to as a service registry. NIST provides the following strategies for service registry configuration:⁵⁵

- Service registry capabilities should be provided through servers that are either dedicated or part of a service mesh⁵⁶ architecture.
- Service registry services should be in a network that has been configured with certain quality of service parameters to ensure its availability and resilience.
- Communication between an application service and a service registry should occur through a secure communication protocol (e.g., hypertext transfer protocol secure or transport layer security).⁵⁷
- Service registries should be validated to ensure that only legitimate services perform the registration, refresh operations, and query the database to discover services.
- Service registration and service deregistration functions should follow the principles of bounded context and loose coupling. Management should set parameters for the processes of registration to and deregistration from the service registry.⁵⁸
- If a third-party registration pattern⁵⁹ is implemented, registration and deregistration should only take place after the performance of a health check⁶⁰ on the application service.
- A distributed service registry should be deployed for large microservices applications, and data consistency should be maintained among multiple service registry instances.⁶¹

⁵⁵ [Ibid.](#)

⁵⁶ NIST SP 800-204A, [Building Secure Microservices-Based Applications Using Service-Mesh Architecture](#), defines “service mesh” as “a distributed computing middleware that optimizes communications between application services.”

⁵⁷ NIST defines “transport layer security” as “an authentication and security protocol widely implemented in browsers and web servers.” For more information, refer to NIST SP 800-63-3, [Digital Identity Guidelines](#).

⁵⁸ As noted in NIST SP 800-204, [Security Strategies for Microservices-Based Application Systems](#), the application service should not have tight coupling with an infrastructure service, such as a service registry service, and service self-registration and deregistration patterns should be avoided. When an application service crashes or is running but unable to handle requests, its inability to perform deregistration affects the integrity of the whole process. Therefore, registration and deregistration of an application service should be enabled using a third-party registration pattern, and the application service should be restricted to querying the service registry for service location information as described under the client-side discovery pattern.

⁵⁹ For purposes of this discussion, a pattern refers to repeatable processes to assist in the standardized design, monitoring, and maintenance of microservices.

⁶⁰ A health check or health testing is testing in an implementation immediately before or during normal operation to determine that the implementation continues to perform as implemented and as validated. Refer to [Law Insider’s Dictionary](#).

⁶¹ Refer to NIST SP 800-204, [Security Strategies for Microservices-Based Application Systems](#), for more information on distributed service registry, which involves several service registry agents cooperating to provide controlled access to resources. Distribution of the registry agents results in improved availability, higher concurrency, better response times to user queries, and enhanced flexibility.